==========

FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION

==========

1. INTRODUCTION

Project Title : IMovies - Movie Ticket Booking Made Simple

Team Members : **Individual project**

HARSHITA PATIDAR - Full Stack Developer

(Frontend, Backend, Database, Authentication, Documentation)

University - MEDICAPS UNIVERSITY

2. PROJECT OVERVIEW

Purpose:

IMovies is an online movie ticket booking platform that improves the cinema experience. Users can browse movies, select showtimes and reserve seats with real-time availability tracking. The project is built with the MERN stack (MongoDB, Express.js, React.js, Node.js) using a client-server architecture for efficient data management and smooth performance.

Goals:

- Make movie ticket booking simple and fast.
- Prevent double booking of the same seat.
- Provide secure user accounts and personal booking history.

Key Features:

1. User registration and login with secure authentication (JWT).
2. Browse all movies, search by title and filter by genre.
3. Movie details page with upcoming showtimes grouped by date.
4. Interactive seat map (rows A-H, seats 1-10) showing available,
   selected and booked seats.
5. Booking management: confirm a booking, view "My Bookings",
   cancel a booking (seats are released automatically).
6. Atomic seat reservation so two users can never book the
   same seat at the same time.
7. Responsive design using Bootstrap (works on mobile and desktop).
8. Protected pages: booking and My Bookings need login.

3. ARCHITECTURE

Overall flow:

```
React (Vite + Bootstrap)  --/api-->  Express.js  -->  Mongoose  -->  MongoDB
      |                                |
      |  JWT stored in localStorage      |  JWT middleware protects
      |  and sent in request header      |  the /api/bookings routes
```

Frontend (React.js):

- Built with React 18 and Vite; styled with Bootstrap 5 and custom CSS.
- React Router handles pages: Home, Movie Details, Booking,
  My Bookings, Login, Register.
- AuthContext (React Context API) stores the logged-in user and
  provides login, register and logout functions.
- ProtectedRoute redirects non-logged-in users to the Login page.
- A small api.js helper uses fetch with async/await and attaches

the JWT automatically to every request.

- Reusable components: Navbar, MovieCard, SeatGrid.

Backend (Node.js and Express.js):

- Express server in server.js with CORS and JSON body parsing.
- Routes are split by feature: auth, movies, shows, bookings.
- auth middleware verifies the JWT before allowing booking actions.
- Mongoose connects to MongoDB; async/await is used everywhere.
- Passwords are hashed with bcryptjs before saving.

Database (MongoDB with Mongoose):

Collections and schema:

users

- name (String, required)
- email (String, required, unique, lowercase)
- password (String, hashed, hidden from queries)
- createdAt, updatedAt

movies

- title (String, required)
- description (String)
- genre (Array of String)
- language (String)
- duration (Number, minutes)
- rating (Number)
- poster (String, image URL)

shows

- movie (ObjectId -> movies)
- theatre (String)

- startTime (Date)
- price (Number)
- rows (Number, default 8), cols (Number, default 10)
- bookedSeats (Array of String, e.g. ["A1", "B4"])

bookings
- user (ObjectId -> users)
- show (ObjectId -> shows)
- seats (Array of String)
- totalPrice (Number)
- status (CONFIRMED or CANCELLED)

Relationships:
movies 1 --- many shows
users 1 --- many bookings
shows 1 --- many bookings

4. SETUP INSTRUCTIONS

Prerequisites:
- Node.js v16 or above (https://nodejs.org)
- npm v8 or above (comes with Node.js)
- MongoDB (local MongoDB Community Server) OR a free MongoDB Atlas cloud cluster
- Visual Studio Code (recommended editor)
- Git
- Postman (optional, for testing APIs)
- Browser: Google Chrome / Firefox (latest)

Installation:

Step 1: Get the project

git clone <your-github-repository-link>

cd imovies

(or unzip the project folder and open it in VS Code)

Step 2: Install backend dependencies

cd backend

npm install

Step 3: Create the environment file

Windows : copy .env.example .env

Mac/Linux: cp .env.example .env

Open backend/.env and set:

PORT=5000

MONGO_URI=mongodb://127.0.0.1:27017/imovies

JWT_SECRET=your_long_random_secret_text

JWT_EXPIRES_IN=7d

For MongoDB Atlas use your connection string, for example:

MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/imovies?retryWrites=true&w=majority

Step 4: Load sample data (6 movies and their shows)

npm run seed

Step 5: Install frontend dependencies

cd ../frontend

npm install

5. FOLDER STRUCTURE

```
imovies/
|-- README.md
|-- .gitignore
|-- backend/                (Server - Node.js + Express.js)
|   |-- server.js           Entry point: Express app and routes
|   |-- seed.js             Inserts sample movies and shows
|   |-- package.json
|   |-- .env.example        Environment variable template
|   |-- config/
|   |   `-- db.js           MongoDB connection
|   |-- middleware/
|   |   `-- auth.js         JWT verification middleware
|   |-- models/
|   |   |-- User.js
|   |   |-- Movie.js
|   |   |-- Show.js
|   |   `-- Booking.js
|   `-- routes/
|       |-- auth.js         register, login, me
|       |-- movies.js       list, details, shows of a movie
|       |-- shows.js        show details and booked seats
|       `-- bookings.js     create, list, cancel bookings
`-- frontend/               (Client - React.js)
    |-- index.html
    |-- package.json
    |-- vite.config.js      Dev server and /api proxy to backend
    `-- src/
        |-- main.jsx        App entry, Router, Bootstrap import
        |-- App.jsx         Route definitions
```

```
|-- api.js            fetch helper with JWT header
|-- styles.css        Custom styles (seat map, hero)
|-- context/
|   `-- AuthContext.jsx
|-- components/
|   |-- Navbar.jsx
|   |-- MovieCard.jsx
|   |-- SeatGrid.jsx
|   `-- ProtectedRoute.jsx
`-- pages/
    |-- Home.jsx
    |-- MovieDetails.jsx
    |-- Booking.jsx
    |-- MyBookings.jsx
    |-- AuthForm.jsx
    |-- Login.jsx
    `-- Register.jsx
```

6. RUNNING THE APPLICATION

Open two terminals in VS Code.

Backend (Terminal 1):

```
cd backend
npm run dev        (auto-restarts on changes)
or: npm start      (plain start)
Runs at: http://localhost:5000
```

Frontend (Terminal 2):

```
cd frontend
```

npm run dev

Runs at: http://localhost:5173

Note: The frontend uses Vite, so its start command is "npm run dev"

(not "npm start"). Open http://localhost:5173 in the browser.

Keep both terminals running while using the application.

7. API DOCUMENTATION

Base URL: http://localhost:5000/api

Auth column: "Yes" means header Authorization: Bearer <token> is required.

--- Authentication ---

POST /auth/register Auth: No

Body:

{ "name": "Asha", "email": "asha@mail.com", "password": "secret123" }

Response 201:

{ "token": "<jwt>", "user": { "id": "...", "name": "Asha", "email": "asha@mail.com" } }

Errors: 400 missing fields, 409 email already registered

POST /auth/login Auth: No

Body:

{ "email": "asha@mail.com", "password": "secret123" }

Response 200:

{ "token": "<jwt>", "user": { "id": "...", "name": "Asha", "email": "asha@mail.com" } }

Errors: 401 invalid email or password

GET /auth/me Auth: Yes

Response 200:

{ "id": "...", "name": "Asha", "email": "asha@mail.com" }

Errors: 401 token missing or invalid

--- Movies ---

GET /movies Auth: No

Query (optional): q = search text, genre = genre name

Example: /movies?q=galaxy&genre=Sci-Fi

Response 200:

[{ "_id": "...", "title": "Galaxy Rangers", "genre": ["Sci-Fi","Action"],

"duration": 142, "rating": 8.4, "language": "English", "poster": "..." }]

GET /movies/:id Auth: No

Response 200: one movie object

Errors: 404 movie not found, 400 invalid id

GET /movies/:id/shows Auth: No

Response 200 (upcoming shows only):

[{ "_id": "...", "theatre": "PVR Treasure Island",

"startTime": "2026-10-17T13:30:00.000Z", "price": 150, "bookedSeats": [] }]

--- Shows ---

GET /shows/:id Auth: No

Response 200:

{ "_id": "...", "movie": { "title": "...", "poster": "...", "duration": 142 },

"theatre": "...", "startTime": "...", "price": 150,

"rows": 8, "cols": 10, "bookedSeats": ["A1", "A2"] }

Errors: 404 show not found

--- Bookings (all need login) ---

POST /bookings Auth: Yes

Body:

{ "showId": "<show id>", "seats": ["C4", "C5"] }

Response 201:

{ "_id": "...", "user": "...", "show": "...", "seats": ["C4","C5"],

"totalPrice": 300, "status": "CONFIRMED" }

Errors: 400 no seats or more than 8 seats,

409 seat already booked or show already started

GET /bookings/mine Auth: Yes

Response 200: list of the user's bookings with show and movie details

DELETE /bookings/:id Auth: Yes

Response 200: booking with status "CANCELLED" (seats become available again)

Errors: 404 not found, 400 already cancelled

8. AUTHENTICATION

Method: JSON Web Token (JWT), stateless authentication.

How it works:

1. Register / Login: the user sends credentials. The server checks them and returns a signed JWT (valid 7 days, set by JWT_EXPIRES_IN) along with basic user details.
2. Password security: passwords are hashed with bcryptjs (10 salt rounds) before being stored. Plain passwords are never saved and are never returned by the API.
3. Token storage: the React app stores the token in localStorage.
4. Sending the token: api.js adds the header

Authorization: Bearer <token>

to every request.

5. Verification: the protect middleware (middleware/auth.js) verifies the token with JWT_SECRET, loads the user, and attaches it to req.user. If the token is missing, invalid or expired, the server replies 401.
6. Session restore: when the app loads, AuthContext calls /auth/me to restore the logged-in user.
7. Frontend protection: ProtectedRoute sends users who are not logged in to the Login page, then back to the page they wanted.

Authorization:

- All /bookings routes require login.
- A user can only view and cancel their own bookings (queries are filtered by req.user).

9. USER INTERFACE

Pages (insert screenshots or GIFs for each below):

1. Home page

   Hero banner with search box, genre filter buttons, movie poster grid.
2. Register / Login pages

   Simple forms with validation and error messages.

3. Movie details page

   Poster, genre, duration, language, rating, description and showtimes grouped by date with theatre name and price.

4. Seat selection / Booking page

Screen indicator, seat grid (green = available, filled green = selected, grey = booked), live total price and Confirm button.

5. My Bookings page

Cards with movie, theatre, time, seats, amount paid, status and a Cancel button.

UI technologies: Bootstrap 5 grid, cards, forms, buttons, spinners and custom CSS for the seat map. Layout is responsive.

10. TESTING

Testing strategy: manual functional testing plus API testing.

Tools:

- Postman: test each API endpoint (success and error cases).
- Browser (Chrome DevTools): check network calls, console errors and responsive layout.
- MongoDB Compass / Atlas Data Explorer: verify stored data.

Test cases:

Authentication

- Register with a new email -> account created, token returned.
- Register with an existing email -> 409 error.
- Login with wrong password -> 401 error.
- Open /my-bookings without login -> redirected to Login.

Movies

- Search by title and filter by genre return correct results.
- Open a movie with no upcoming shows -> "No upcoming shows".

Booking

- Select seats and confirm -> booking appears in My Bookings and seats show as booked on the seat map.
- Two users book the same seat at the same time -> only one succeeds, the other gets a 409 error.
- Try to book more than 8 seats -> rejected.
- Cancel a booking -> status CANCELLED and seats become available.

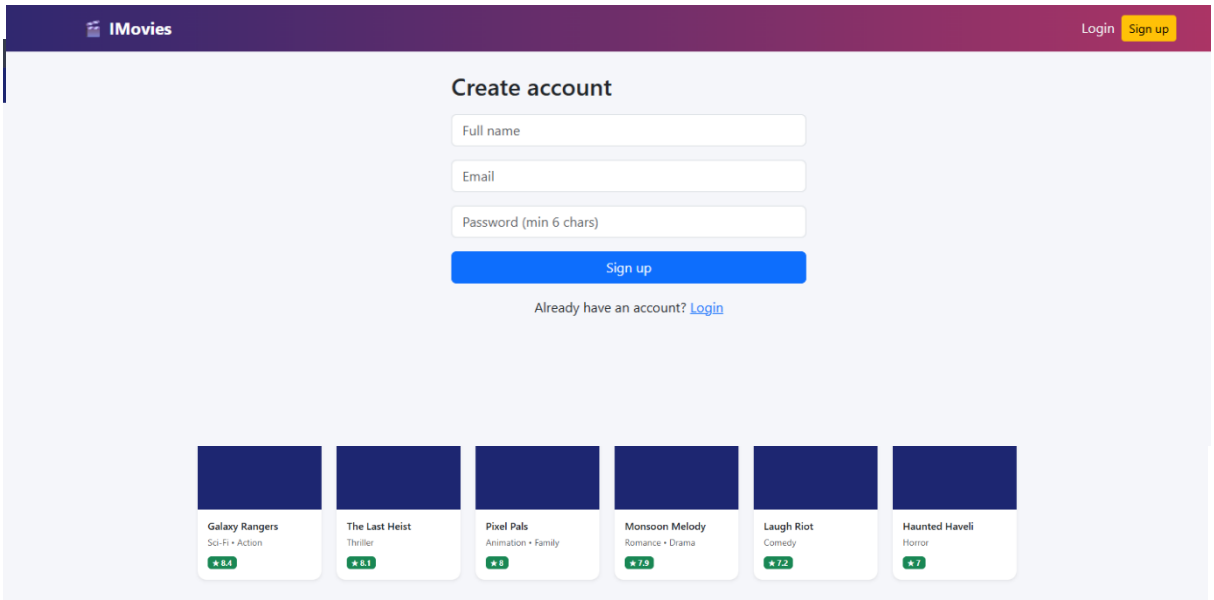
Future testing improvements: automated tests with Jest and Supertest (backend) and React Testing Library (frontend).

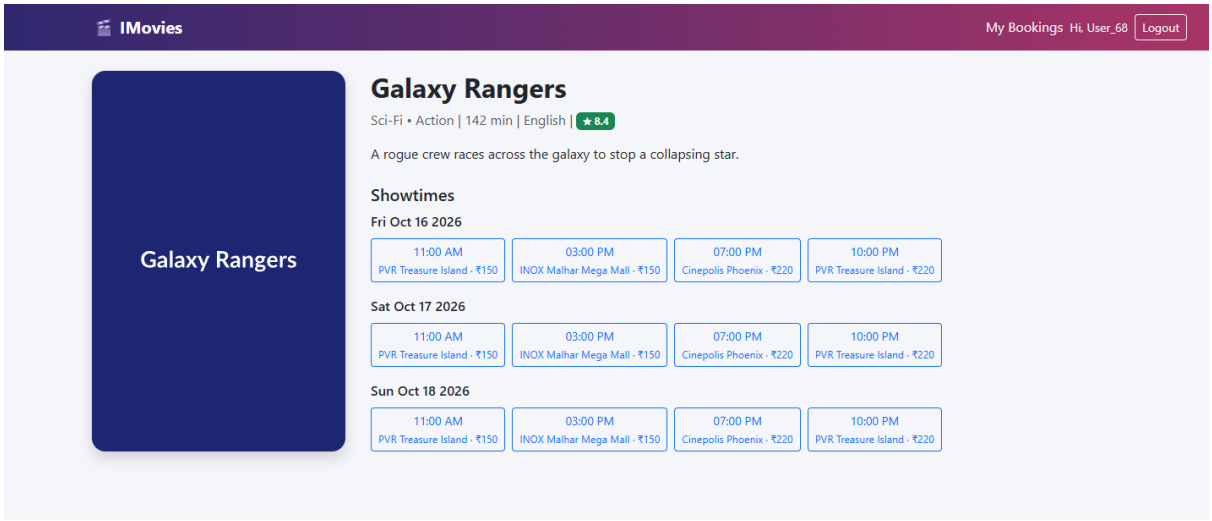
11. **SCREENSHOTS OR DEMO**

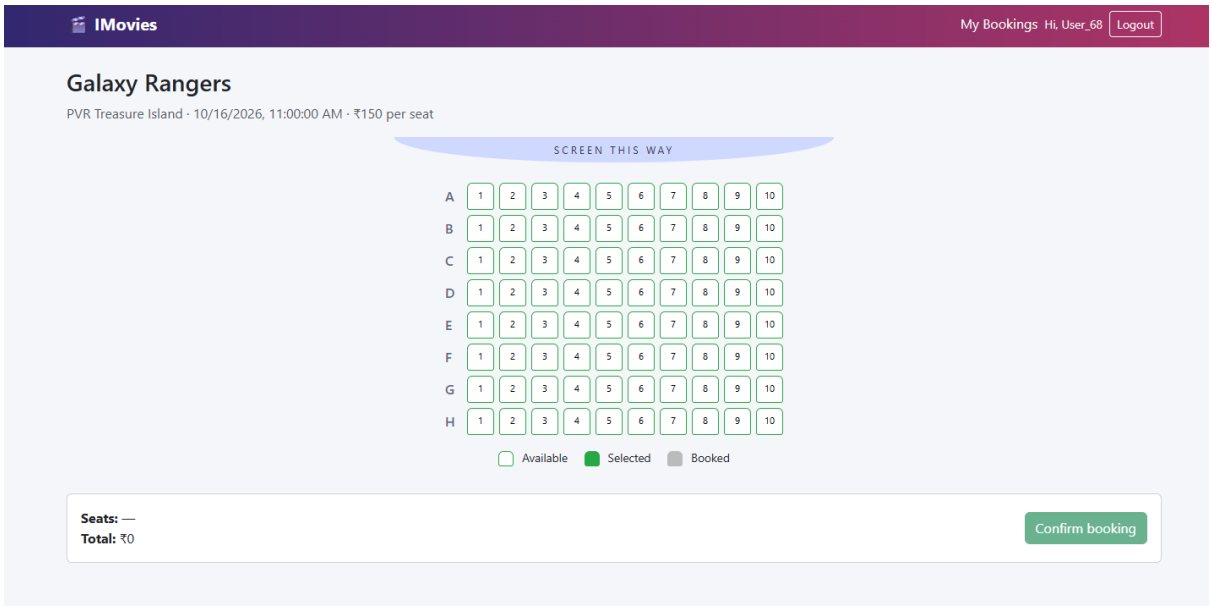
Demo link : https://imovies-1.onrender.com

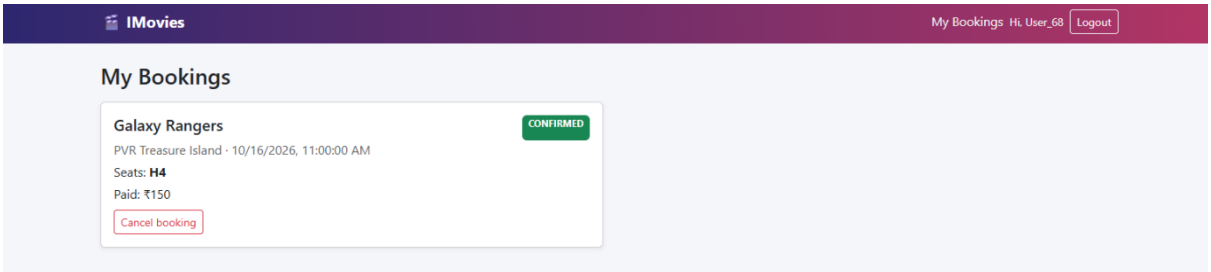
GitHub link : https://github.com/harshi-240/imovies.git

Screenshots :




IMovies
My Bookings Hi, User_68 Logout
Galaxy Rangers
Galaxy Rangers
Sci-Fi • Action | 142 min | English | ★ 8.4
A rogue crew races across the galaxy to stop a collapsing star.
Showtimes
Fri Oct 16 2026
11:00 AM PVR Treasure Island · ₹150
03:00 PM INOX Malhar Mega Mall · ₹150
07:00 PM Cinepolis Phoenix · ₹220
10:00 PM PVR Treasure Island · ₹220
Sat Oct 17 2026
11:00 AM PVR Treasure Island · ₹150
03:00 PM INOX Malhar Mega Mall · ₹150
07:00 PM Cinepolis Phoenix · ₹220
10:00 PM PVR Treasure Island · ₹220
Sun Oct 18 2026
11:00 AM PVR Treasure Island · ₹150
03:00 PM INOX Malhar Mega Mall · ₹150
07:00 PM Cinepolis Phoenix · ₹220
10:00 PM PVR Treasure Island · ₹220


IMovies
My Bookings Hi, User_68 Logout
Galaxy Rangers
PVR Treasure Island · 10/16/2026, 11:00:00 AM · ₹150 per seat
SCREEN THIS WAY
A 1 2 3 4 5 6 7 8 9 10
B 1 2 3 4 5 6 7 8 9 10
C 1 2 3 4 5 6 7 8 9 10
D 1 2 3 4 5 6 7 8 9 10
E 1 2 3 4 5 6 7 8 9 10
F 1 2 3 4 5 6 7 8 9 10
G 1 2 3 4 5 6 7 8 9 10
H 1 2 3 4 5 6 7 8 9 10
Available Selected Booked
Seats: —
Total: ₹0
Confirm booking


IMovies
My Bookings Hi, User_68 Logout
My Bookings
Galaxy Rangers
CONFIRMED
PVR Treasure Island · 10/16/2026, 11:00:00 AM
Seats: H4
Paid: ₹150
Cancel booking

Suggested deployment:

- Database : MongoDB Atlas (free tier)
- Backend : Render (set the .env values as environment variables)
- Frontend : Vercel or Netlify (point the API base URL to the deployed backend)

12. KNOWN ISSUES

---

1. No real payment gateway; booking is confirmed without payment.
2. Seats are reserved immediately on confirmation; there is no temporary seat "hold" timer while a user is choosing seats.
3. The seat map shows the state at page load. If another user books meanwhile, it updates only after the booking attempt fails or the page is refreshed (the server still blocks double booking).
4. Poster images are placeholders loaded from an external URL and need internet access.
5. JWT is stored in localStorage, which is simple but less secure than httpOnly cookies.
6. No password reset, email verification or admin panel yet.
7. Automated test suite is not included yet.
8. "npm audit" may report vulnerabilities in dependencies; do not use "npm audit fix --force" without testing.

13. FUTURE ENHANCEMENTS

- Payment gateway integration (Razorpay / Stripe).
- Temporary seat lock with a countdown timer during checkout.
- Live seat updates using WebSockets (Socket.io).
- E-ticket with QR code, sent by email (Nodemailer) or SMS.

- Admin dashboard to add movies, theatres, shows and view reports.
- Password reset and email verification.
- Movie reviews, ratings and watchlist.
- Theatre and city selection with location-based search.
- Different seat categories (Silver, Gold, Recliner) and dynamic pricing.
- Coupons and offers.
- Automated tests and CI/CD pipeline.
- Progressive Web App / mobile app version.

==========

END OF DOCUMENTATION

==========